Multi-tenant background agents

Julen Gamboa. Principal AI Engineer take-home. 14 Oct 2026.

Solution design

The design uses three main parts: a scheduler, a container, and the egress proxy.

1. **Scheduler.** It picks which agent runs next, by deadline, within provider, tenant and host limits. It starts, resumes and escalates runs.
2. **Container.** Each run gets its own locked-down container, described under Sandbox.
3. **Egress proxy.** Every outbound call goes through it. It maps the run token to a tenant, enforces the allowlist, checks the budget, injects the real key and writes the journal entry; the agent switches model on failure. Two instances run side by side and share only Postgres.

Stack

Postgres 16 holds the step journal, outbox, memberships and eval records, with forced row-level security. Python 3.12 runs the scheduler, proxy and agent loop. Docker runs each agent container and Postgres. `make drill` starts Postgres, applies migrations and runs every failure scenario against recorded model calls.

The drill replays recorded calls, injects an error and a rate limit, marks both `simulated`, and finishes on the recorded fallback.

Scheduling 200 agents

Agents give a deadline, not a start minute, so 200 briefs due at 07:30 spread across 90 minutes instead of all firing at 06:00. Dispatch is earliest deadline first, behind these limits:

- **Provider:** requests and tokens per minute. When full, a run waits or switches model. A self-hosted model is one more provider, sized by its batch slots.
- **Tenant:** each tenant has a concurrency limit, and it says how many of its runs happen at once.
- **Host:** 8 containers, each with 0.5 CPU and 512 MB of RAM.
- **Rate limits:** runs start on the API. On a 429 the agent retries with exponential backoff and jitter; if throttling continues, it falls back to the next model in the config. Each provider's token and request quota is split across tenants, and runs are capped at each provider's concurrency.

If the scheduler predicts a missed deadline, it escalates to a person.

Sandbox

Each run gets a new container: a regular user on a read-only filesystem, a 64 MB scratch disk, no extra capabilities, no privilege escalation, the default seccomp profile, and process, CPU and memory limits. `py.exec` runs as a subprocess in the same container.

The container can talk to the proxy and nothing else. Its run token names the run but is not a credential. The proxy ties the token to the container's IP, reads the tenant and allowlist from it, and adds the real provider key on the way out. Another tenant's mail, a `CONNECT` tunnel and a call to `127.0.0.1:18894` through the proxy each return a denial and log `policy_denied`.

Failure and resume

Each step is keyed by `run_id`, `step_no` and a hash of the messages and the tools; the model is not in the key, so a run can switch model mid-flight. The proxy writes `STARTED` before calling out and `COMPLETED` or `FAILED` before replying. Postgres has its own container, so a killed worker leaves the journal intact. A run is `SUCCEEDED` only if its artifact cites real message IDs from its own tenant; no artifact means `FAILED`.

Every failure ends in one of three logged outcomes:

1. **Auto-resume.** A killed worker resumes in a new container with the same `run_id` once the lease frees, skipping completed steps. Tool errors retry 3 times, then go to the model. A hang is cancelled and retried once. Three provider errors, or throttling past backoff, switch to the next model and log `provider_fallback`. Sandbox denials are logged and the run continues.

2. **Rerun with logged cost.** A step that died while `STARTED` reruns and logs `cost_uncertain`, since the provider may have charged already.
3. **Hand to a person.** A loop (the same call 3 times) stops as `STOPPED_LOOP`; a call that would pass a limit is skipped and the run stops as `STOPPED_BUDGET`. Both save progress first. A person also takes over on the third lease expiry (after two resumes), when every model is down, when a retry fails again, on a third sandbox denial, or before a missed deadline.

Tenants, evals, memory, panel, drafts

Tenants. Each user has one tenant membership with a role: viewer, approver or admin. There is no signup flow; a membership is a row. Isolation has two layers, and both hold if an agent is compromised: the proxy checks the run token's tenant scope on every call and alone holds the provider key, and Postgres row-level security scopes every query to that tenant, so an unscoped query returns no rows. In the drill, the approver is a membership row.

Evals. Each agent has a baseline of 20 or more recorded cases with expected outputs, including the injection email; the synthetic mailbox (about 30 emails across four tenants) is the brief agent's workload, not the baseline. A temperature-0 judge model scores each case 1 to 5. Hard checks must pass: tenant-only data, no unflagged external draft, matching spreadsheet figures. A new prompt or model gets a new config hash and reruns the same baseline, keeping scores comparable. `evals run` writes an `eval_record` only if hard checks pass and the mean drops 0.3 or less; the first record uses hard checks only.

Memory. Memory is Postgres rows keyed to a tenant and an agent: a cursor marking the last item processed, facts saved with `memory.write`, and summaries of past runs. Row-level security scopes every read and write to the run token's tenant. Each run loads the facts and summaries into the prompt as quoted data, and `mail.list` starts after the cursor. The cursor moves only on `SUCCEEDED`, so a crashed run retries from the same position.

Panel (design only). Three models answer blind, then challenge each other's claims with evidence over two rounds. The parent returns agreed claims, contested ones with how each was settled, and any minority view kept, because agreement is not proof.

Drafts. Agents write drafts but cannot send. Sending is a separate action that needs an approver in the same tenant; it sets `Message-ID` from the draft ID and moves the outbox from `SENDING` to `SENT`, and after a crash it checks by that ID, so a replay never sends twice. A draft that goes outside the tenant's domains or attaches documents is flagged and needs its own approval. Mail and memory enter the prompt as quoted data, not instructions, so a malicious email gets no further than a flagged draft.

Not building

Live mailbox and document connectors, a web UI, OIDC, a replica, multi-host scheduling and the panel. The synthetic service has the connector interface, so a real connector drops in.

No gVisor or Firecracker microVMs: the build has two days, and microVMs need KVM on a Linux host. The trade-off is a shared kernel: a kernel exploit in one container could escape to the host and reach other tenants.

The weakest part, and how to harden it

The proxy. It holds every credential, the tenant map and the budget ledger. Two instances survive a crash, but they run the same code, so one bug breaks tenancy for every run. Hardening takes about one week. Each step starts as a failing property test:

1. **Detect leaks**: canary IDs per tenant; a fuzzer over token, tenant and path.
2. **Stop sharing the kernel**: a Firecracker microVM per run from a warm pool.
3. **No long-lived secrets**: a vault and a broker issuing five-minute, one-tenant tokens.
4. **Separate failure domains**: gateway, broker and ledger over mutual TLS, each failing closed.
5. **A journal that survives a host**: a synchronous Postgres replica and lease fences.